

How I Deployed My Angular App on Azure

Azure DevOps CI/CD Pipeline + Azure App Service + All the Errors I Fixed

A step-by-step guide written as I actually did it

Let me explain what we are doing here

So basically the whole deployment is split into two parts. First part is CI which means Continuous Integration and this happens in Azure DevOps. Second part is CD which means Continuous Deployment and that happens in Azure Portal (portal.azure.com). These are two different websites, don't get confused.

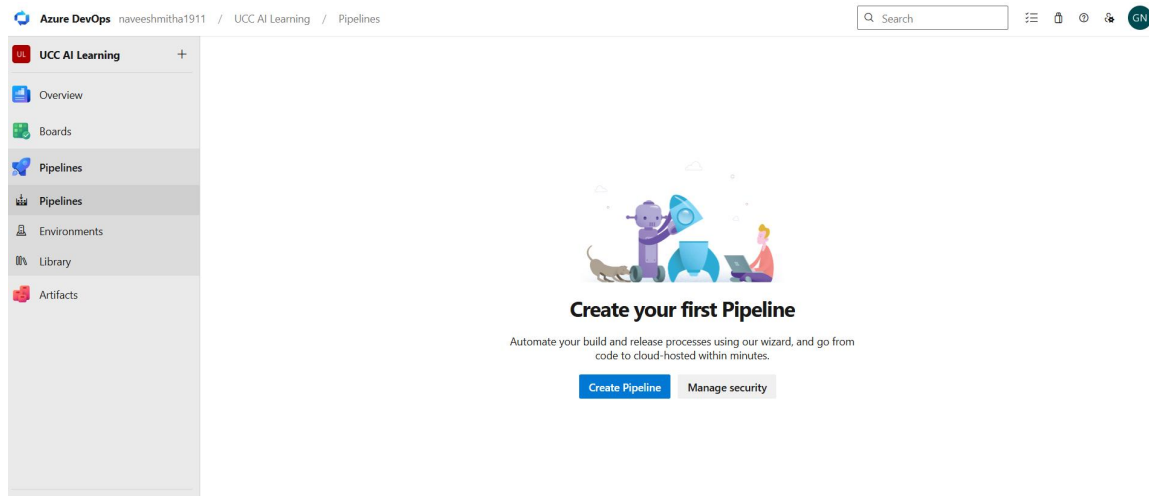
CI (Azure DevOps): This is where we create a pipeline that connects to our GitHub code. Whenever we push code to the main branch, it will automatically build our Angular project. This just builds the code, it does not make the website live yet.

CD (Azure Portal): This is where we create the actual server on Azure cloud, create an App Service (basically a hosted website), and connect it to our GitHub so it picks up the built code and makes our website live on the internet.

Both steps are needed. CI builds the code, CD hosts it. Now let me walk through everything exactly as I did it.

Step 1 — Logging into Azure DevOps

First I logged in to Azure DevOps. The URL is dev.azure.com. I used my Microsoft account to sign in. Once logged in I got the Azure DevOps dashboard page.

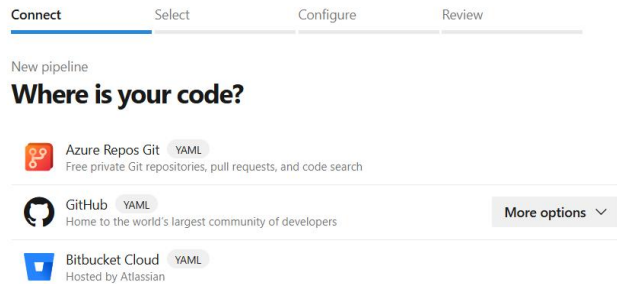


This is the Azure DevOps dashboard after logging in

What is Azure DevOps? Azure DevOps is a platform by Microsoft where you manage your code pipelines, builds, and deployments. Think of it like a control center. It connects to your GitHub code and can automatically build and test your project every time you make a change.

Step 2 — Creating the Pipeline (CI Part)

From the dashboard the first thing I needed to do was create a pipeline. The pipeline is basically a set of instructions that tells Azure what to do with our code when we push it to GitHub. To create it I clicked on Pipelines in the left sidebar.



Pipelines section in Azure DevOps - this is where we start

Then I clicked New Pipeline. It takes you through 4 steps at the top - Connect, Select, Configure, Review. It goes step by step.

✓ Connect **Select** Configure Review

New pipeline

Select a repository

Filter by keywords

My repositories

 greeshugithub/ucc-learning-platform
2h ago

 greeshugithub/ucc-learning
Apr 26

 greeshugithub/sis
Sep 6, 2025

 greeshugithub/angular fork
Aug 21, 2025

 greeshugithub/greeshugithub
Jun 12, 2025

 greeshugithub/Tic-tac-toe
Jun 10, 2025

 greeshugithub/Portfolio
May 24, 2025

The 4-step pipeline creation wizard - Connect, Select, Configure, Review

Connect Step — Connecting to GitHub

In the Connect step I selected GitHub. Azure DevOps then asked me to authorize and verify with my email. It basically redirects to GitHub and asks for permissions so Azure can read my repositories. I approved it and it connected successfully. Once done it came back to Azure DevOps.

✓ Connect ✓ Select **Configure** Review

New pipeline

Configure your pipeline

 **Node.js**
Build a general Node.js project with npm.

 **Node.js Express Web App to Linux on Azure**
Build a Node.js Express app and deploy it to Azure as a Linux web app.

 **Node.js with Vue**
Build a Node.js project that uses Vue.

 **Node.js with webpack**
Build a Node.js project using the webpack CLI.

 **Node.js with React**
Build a Node.js project that uses React.

 **Node.js React Web App to Linux on Azure**
Build a Node.js React app and deploy it to Azure as a Linux web app.

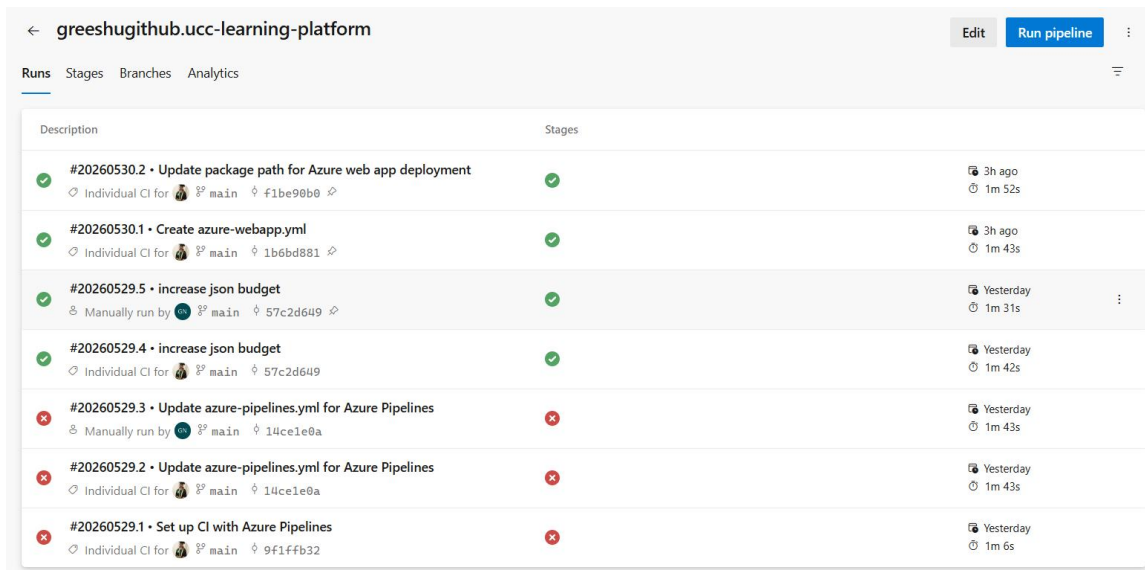
 **Node.js with Angular**
Build a Node.js project that uses Angular.

 **Starter pipeline**

GitHub connected and asking to select which repository

Select Step — Picking My Repository

After connecting GitHub it showed me all my repositories. I selected the Angular project repo that I wanted to deploy. Done.



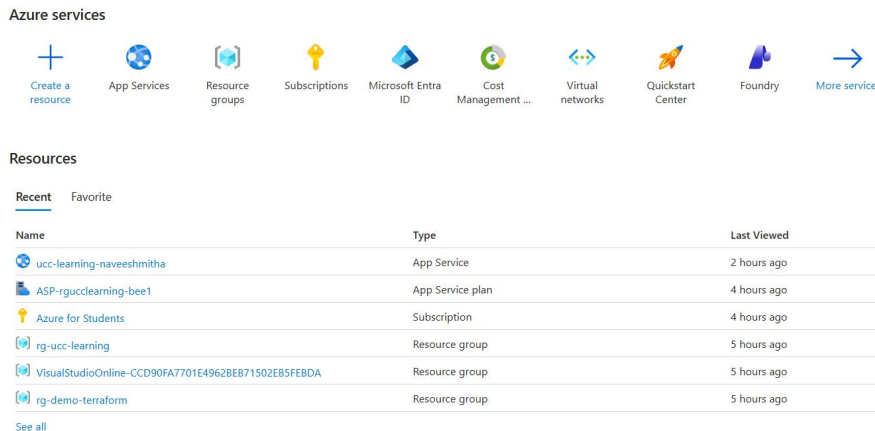
The screenshot shows the 'Runs' tab of an Azure Pipelines pipeline named 'greeshugithub.ucc-learning-platform'. The table lists several runs with their descriptions, stages, and completion times. The first four runs are successful, while the last three are failed.

Description	Stages	Time
#20260530.2 • Update package path for Azure web app deployment Individual CI for [user] main 1b6bd881	✓	3h ago 1m 52s
#20260530.1 • Create azure-webapp.yml Individual CI for [user] main 1b6bd881	✓	3h ago 1m 43s
#20260529.5 • increase json budget Manually run by [user] main 57c2d649	✓	Yesterday 1m 31s
#20260529.4 • increase json budget Individual CI for [user] main 57c2d649	✓	Yesterday 1m 42s
#20260529.3 • Update azure-pipelines.yml for Azure Pipelines Manually run by [user] main 14ce1e0a	✗	Yesterday 1m 43s
#20260529.2 • Update azure-pipelines.yml for Azure Pipelines Individual CI for [user] main 14ce1e0a	✗	Yesterday 1m 43s
#20260529.1 • Set up CI with Azure Pipelines Individual CI for [user] main 9f1ffb32	✗	Yesterday 1m 6s

Selecting my Angular project repository from GitHub

Configure Step — Selecting Angular + Node.js

Now it asked what type of project I am building. Since my project is Angular which uses Node.js, I selected the Node.js with Angular option. This automatically sets up the right YAML template for me.



The screenshot shows the 'Azure services' section with various service icons like 'Create a resource', 'App Services', 'Resource groups', etc. Below it, the 'Resources' section is visible, showing a table of recent resources.

Name	Type	Last Viewed
ucc-learning-naveeshmitha	App Service	2 hours ago
ASP-rguclearning-bee1	App Service plan	4 hours ago
Azure for Students	Subscription	4 hours ago
rg-ucc-learning	Resource group	5 hours ago
VisualStudioOnline-CCD90FA7701E49628EB71502E85FEBDA	Resource group	5 hours ago
rg-demo-terraform	Resource group	5 hours ago

Selected "Node.js with Angular" as the configuration template

Why Angular + Node.js? Angular projects need Node.js to build because the Angular CLI (ng build) runs on Node.js. So Azure needs Node.js installed on the build machine first before it can run the Angular build commands.

Step 3 — The YAML Pipeline Code

After selecting the template it automatically generated the YAML code for me. YAML is basically the configuration file that tells the pipeline what commands to run. This is what I got:

```
# Node.js with Angular
# Build a Node.js project that uses Angular.

trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- task: NodeTool@0
  inputs:
    versionSpec: '20.x'
    displayName: 'Install Node.js'

- script: |
  npm install -g @angular/cli
  npm install
  ng build --prod
  displayName: 'npm install and build'
```

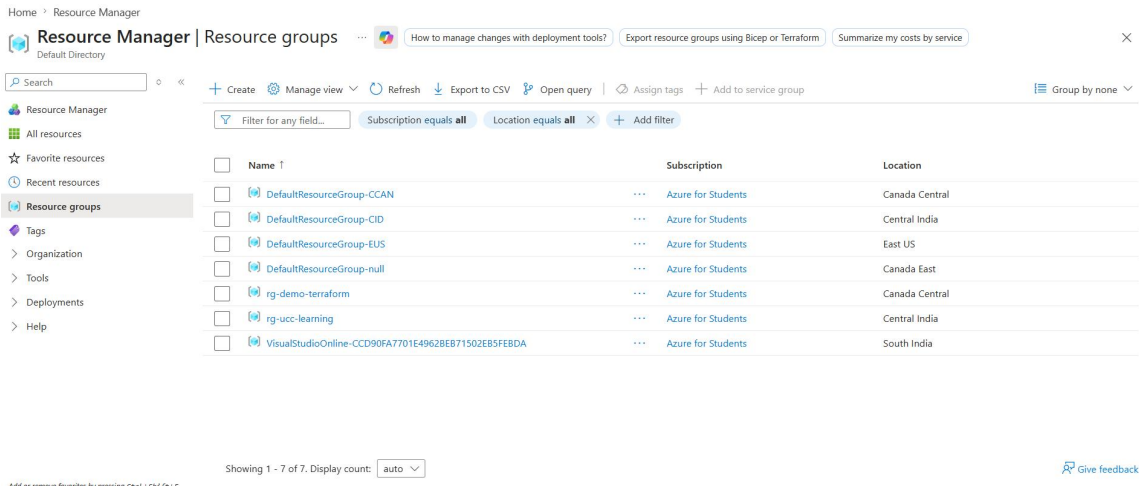
Let me explain each part of this YAML in simple words:

trigger: main — This means the pipeline will run automatically whenever I push code to the main branch on GitHub. So every time I make a commit, it triggers automatically.

pool: vmImage: ubuntu-latest — This is the machine where the build will happen. Azure gives us a virtual machine running Linux (Ubuntu) to run the build on. We don't need to set up anything, Azure handles it.

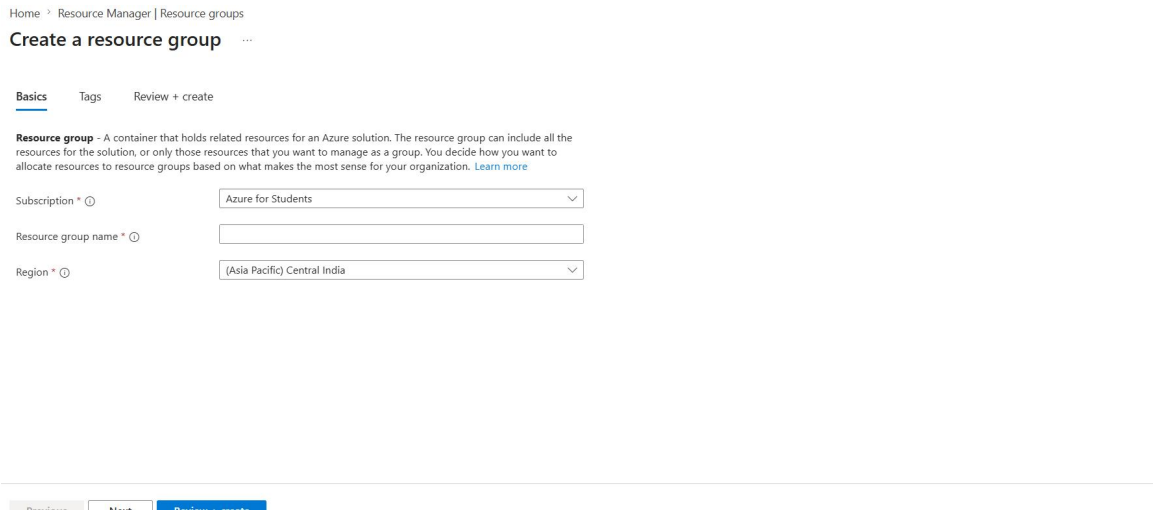
NodeTool@0 with versionSpec 20.x — This installs Node.js version 20 on that Ubuntu machine. Without Node.js we cannot run the Angular build.

script block — This runs three commands one after another. First it installs Angular CLI globally. Then it installs all the project packages (npm install). Then it builds the project for production (ng build --prod).



The YAML code shown in Azure DevOps Review step before saving

After reviewing I clicked Save and Run. This saves the azure-pipelines.yml file into my GitHub repo and also kicks off the first build.



Saved the pipeline and the first run started

Step 4 — I Got an Error! Angular Budget Issue

So when the pipeline ran for the first time the build step failed. I got an error related to budget in angular.json. Let me explain what happened.

Error I Got: ng build --prod failed because the bundle size exceeded the maximum budget. The error said something like bundle size is 2-3 GB but the maximum allowed is 2 MB.

Why did this happen?

In our Angular project there is a file called angular.json. Inside that file there is a budgets setting. This setting controls how big the final compiled files can be. If the output is bigger than that limit, Angular throws an error and refuses to build.

Why does this budget exist? Angular has this budget check to warn you if your app becomes too big to load quickly in a browser. The default limit is usually around 2 MB for the initial bundle. If you have lots of libraries or heavy dependencies, the bundle can easily go above this.

Here is the thing. When I ran ng build locally in VS Code it was working fine. But in the Azure pipeline it was failing. This is because locally I might not have the same strict settings, but in the pipeline the --prod flag makes it treat budget warnings as hard errors.

How I Fixed It

I went into angular.json in VS Code and increased the budget limits. I changed the maximumWarning and maximumError values to something higher so that it would not fail.

Fix: Open angular.json in VS Code, find the budgets section under build > configurations > production, and increase the maximumWarning and maximumError values. For example change maximumError from "2mb" to "5mb" or whatever your app needs.

Mistake I Made: I increased the budget in VS Code and ran ng build locally and it worked. But I forgot to commit and push the changes to GitHub! The pipeline reads from GitHub, not from my local machine. So the pipeline was still seeing the old angular.json with the old budget limits and kept failing.

What I Did: I committed the angular.json changes in VS Code and pushed to the main branch on GitHub. Then I ran the pipeline again and this time it worked.

After committing the fix the pipeline ran successfully. This is the end of the CI part. Now every time I push code to main, Azure will automatically build my Angular project.

Home > Resource Manager

Resource Manager | Resource groups

Export resource groups using Bicep or Terraform | How to manage changes with deployment tools? | Summarize my costs by service

Search

+ Create Manage view Refresh Export to CSV Open query Assign tags Add to service group

Filter for any field... Subscription equals all Location equals all Add filter

☐	Name	Subscription	Location
☐	DefaultResourceGroup-CCAN	Azure for Students	Canada Central
☐	DefaultResourceGroup-CID	Azure for Students	Central India
☐	DefaultResourceGroup-EUS	Azure for Students	East US
☐	DefaultResourceGroup-null	Azure for Students	Canada East
☐	rg-demo-terraform	Azure for Students	Canada Central
☐	rg-ucc-learning	Azure for Students	Central India
☐	VisualStudioOnline-CCD90FA7701E4962BEB71502EB5FEBDA	Azure for Students	South India

Pipeline running again after fixing the budget error and committing

Azure services

Create a resource

Resource groups

App Services

Virtual networks

Quickstart Center

Resources

Recent Favorite

Name

rg-ucc-learning

App Services

+ Create View

- + Web App
- + Web App + Database
- + WordPress on App Service
- + Web App (for Managed Instance)

Learn more with Copilot

Pipeline completed successfully - CI build is working!

Step 5 — Moving to Azure Portal (CD Part Starts Here)

OK so now the CI part is done. The pipeline builds my code automatically. But the website is still not live anywhere. For the website to actually be accessible on the internet I need to host it somewhere. That is where Azure Portal comes in.

Azure Portal is a completely different website from Azure DevOps. The URL is portal.azure.com. This is where we actually create the cloud infrastructure — like the server, the hosting plan, etc.

Create Web App

Basics Database Deployment Networking Monitor + secure Tags Review + create

Try Managed Instances (preview) on Azure App Service: a new option that delivers the platform benefits you rely on today, plus added features and flexibility to help you modernize applications seamlessly

App Service Web Apps lets you quickly build, deploy, and scale enterprise-grade web, mobile, and API apps running on any platform. Meet rigorous performance, scalability, security and compliance requirements while using a fully managed platform to perform infrastructure maintenance. [Learn more](#)

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *
Resource Group *
[Create new](#)

Instance Details

Name .azurewebsites.net

[Review + create](#)

[< Previous](#)

[Next : Database >](#)

Azure Portal dashboard at portal.azure.com

What is the difference between Azure DevOps and Azure Portal? Azure DevOps (dev.azure.com) is for pipelines and code. Azure Portal (portal.azure.com) is for cloud resources like servers, storage, databases. In DevOps we build the code. In Portal we create the place where the code will live and run.

Step 6 — Creating a Resource Group

The first thing I had to do in Azure Portal was create a Resource Group. Before creating the actual web server we need to create a Resource Group first.

What is a Resource Group and why do we need it? A Resource Group is like a folder in Azure. All the related things for your project — the web app, the storage, the database, etc — go inside this folder. It makes it easy to manage everything together. For example if you want to delete your entire project, you just delete the resource group and everything inside it is gone. Also billing and permissions can be managed at the resource group level.

To create it I searched for Resource groups in the Azure Portal search bar and clicked Create.

Home

Create Web App ...

☒ Secure unique default hostname on. [More about this update](#)

Publish * ☒ Code ☐ Container

Runtime stack *

Operating System ☒ Linux ☐ Windows

Region *

i Not finding your App Service Plan? Try a different region or select your App Service Environment.

Pricing plans

App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#)

Linux Plan (Canada Central) ⓘ

Select a resource group before selecting a plan.

Zone redundancy

An App Service plan can be deployed as a zone redundant service in the regions that support it. Your initial instance count will be set based on your zone redundancy configuration. To ensure you'll be able to enable zone redundancy at

Creating a new Resource Group in Azure Portal

I filled in these details:

Subscription: Student (I am using the Azure for Students subscription which gives free credits)

Resource Group Name: rg-uuc-learning

Region: India

Then I clicked Review + Create and it created the resource group. Simple.

The screenshot shows the Azure Portal interface for a Web App named 'ucc-learning-naveeshmitha'. The app is in a 'Stopped' state. The 'Essentials' section shows the resource group 'rg-uuc-learning' and the location 'Sweden Central'. The 'Properties' section shows the subscription ID '8353541d-b1ff-4516-8d43-e1e018d7f25f' and the tags 'Azure for Students'. The 'Health Check' status is 'Cannot determine health check status while app is in a "Stopped" sta'.

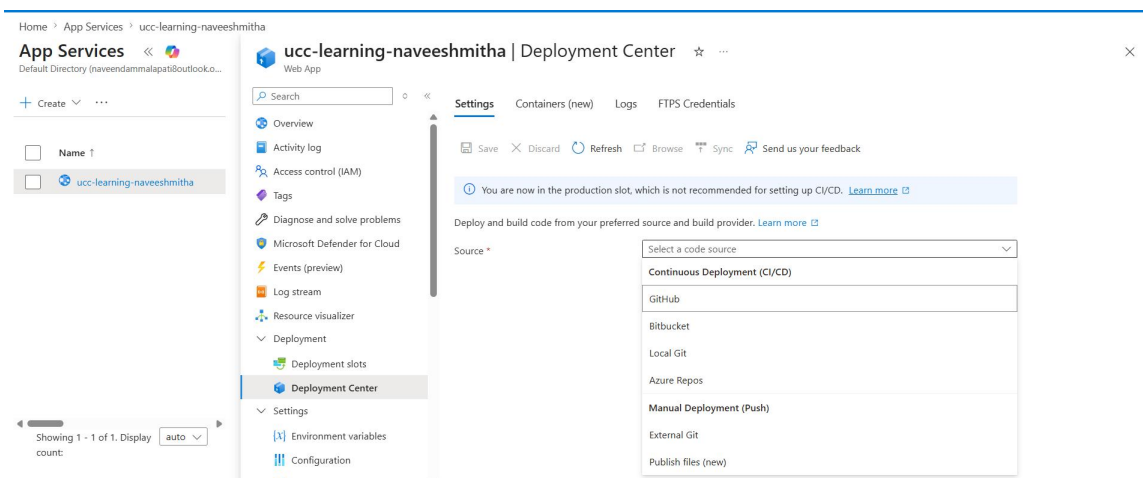
Resource Group created successfully

Step 7 — Creating the Web App (App Service)

Now with the resource group ready, I needed to create the actual web app. This is the thing that will actually run my Angular website. In Azure this is called App Service.

What is App Service? App Service is Azure's platform for hosting web applications. You don't need to set up a virtual machine or install a server manually. Azure manages all of that. You just say "I want a Node.js web app" and Azure gives you a running server with a URL. It handles scaling, SSL, security patches, etc automatically.

From the Portal home page I clicked on App Services and then clicked Create > Web App.



App Service - Web App creation page

I filled in the configuration:

Runtime Stack: Node 22 LTS (because my project runs on Node.js)

Operating System: Linux

Pricing Plan: Free F1 (VERY IMPORTANT - see note below)

Why Free F1 Plan? The Free F1 plan means it does not cost anything. When you are learning and just testing things, always pick Free F1. Even if you forget to delete the resource group later, it will not charge any credits. If you pick a paid plan by mistake and forget to delete it, you will lose your Azure credits.

Region Problem — This Took Me a Long Time

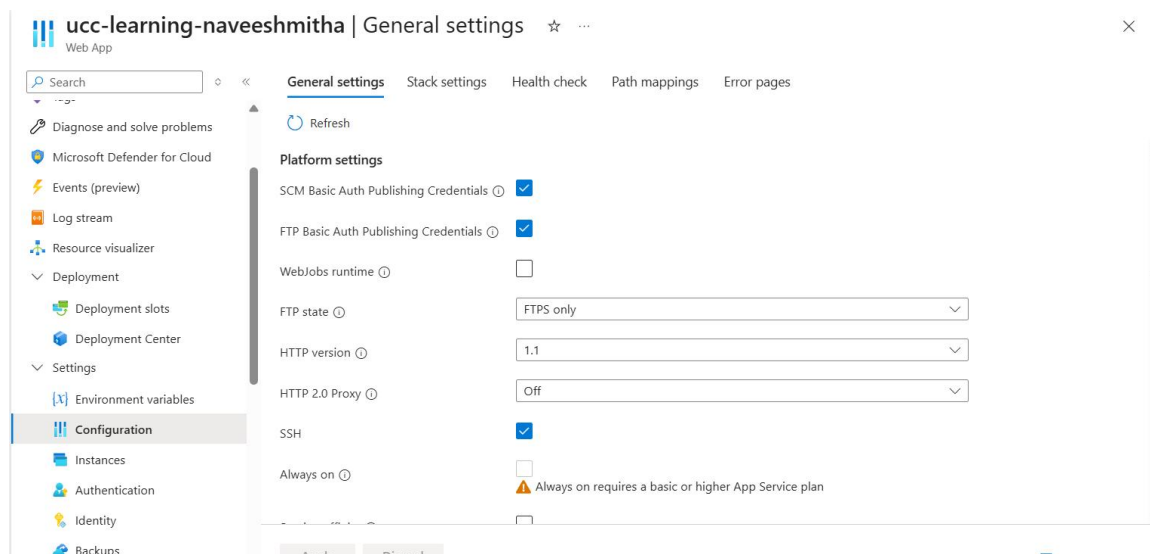
Here is where I got stuck for a long time. When I tried to select the Region for my web app, most regions were not working. I kept trying different regions and getting errors saying the region was not available for my subscription.

Problem: Most Azure regions were not accepted under my Student subscription. Every time I picked a region it showed an error or the web app would not create.

After spending a lot of time I figured out that my Student subscription has policies that restrict which regions I can use. To find the allowed regions I had to go to Settings > Policy > View Assignments in my subscription. From there I could see which regions are actually allowed.

Fix: Go to your Subscription in Azure Portal > Settings > Policy > View Assignments. Check which regions are listed as allowed. Use only those regions. In my case Sweden Central was one of the allowed regions so I selected that. That is why my final URL has swedencentral-01 in it.

Once I selected Sweden Central the web app was created successfully and I got a URL.



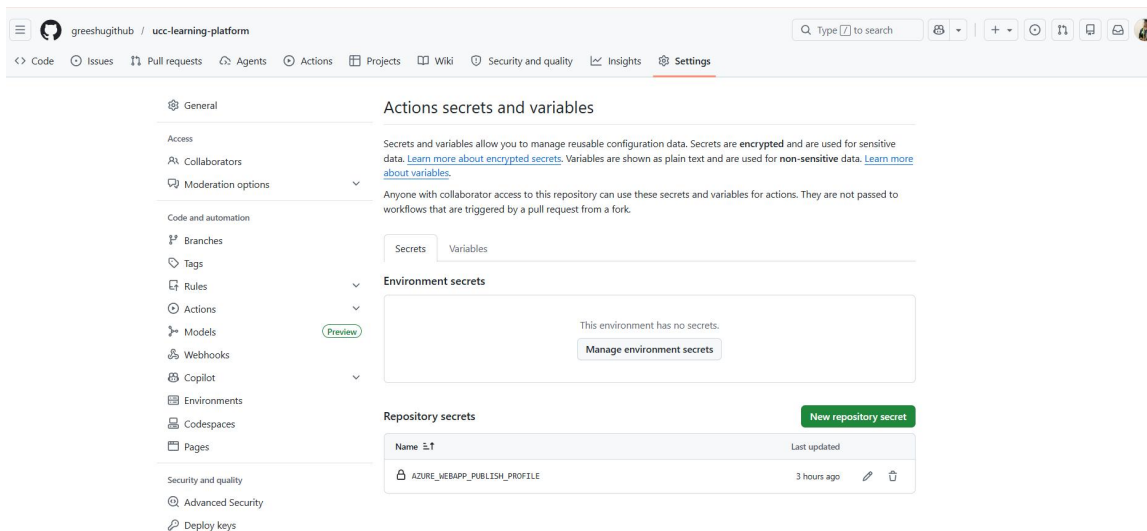
Web App created - but it is just showing the Azure default page, not my actual website yet

Step 8 — Deployment Center — Connecting My GitHub to the Web App

So the web app is created and I have a URL but it is just showing the default Azure page. My actual Angular code is not there yet. To connect my code to this web app I need to use Deployment Center.

What is Deployment Center? Deployment Center is a section inside your App Service where you tell Azure where to pull your code from. You can connect it to GitHub and tell it which repo and which branch to use. Once set up, every time code is pushed to that branch, Azure will automatically pull and deploy it to your web app.

In the App Service sidebar I scrolled down and found Deployment Center. I clicked on it.

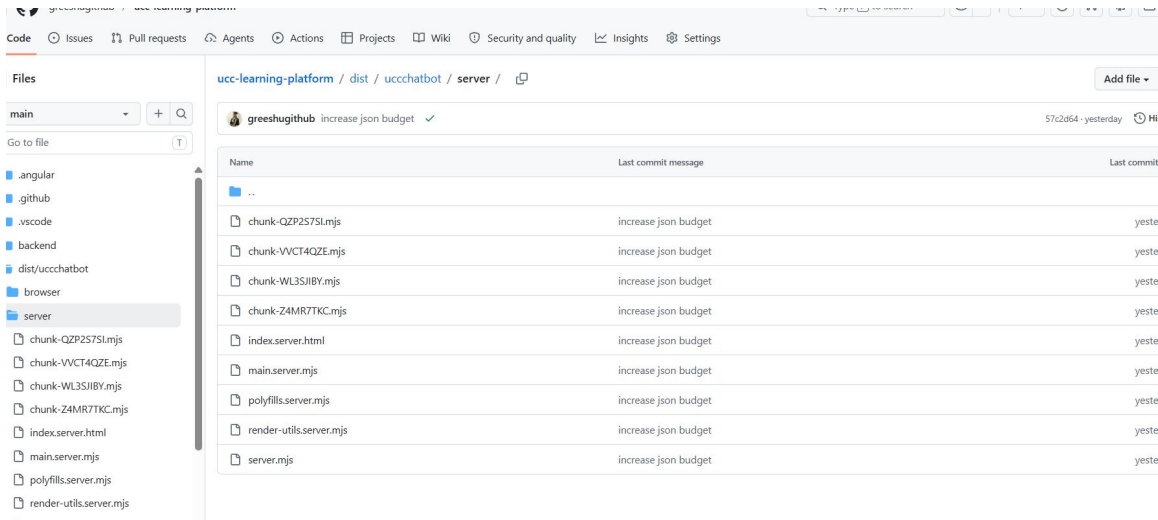


Deployment Center in the App Service left sidebar

In Deployment Center I set the Source as GitHub. Again it asked for Azure authentication and email verification to connect to GitHub. After that was done it asked me for:

- Organization — my GitHub username
- Repository — the Angular project repo
- Branch — main

After filling that in and saving, Azure automatically creates a GitHub Actions workflow YAML file in my repository and starts deploying.



Deployment Center configured with GitHub repo and branch

Step 9 — More Errors! Fixing the Deployment

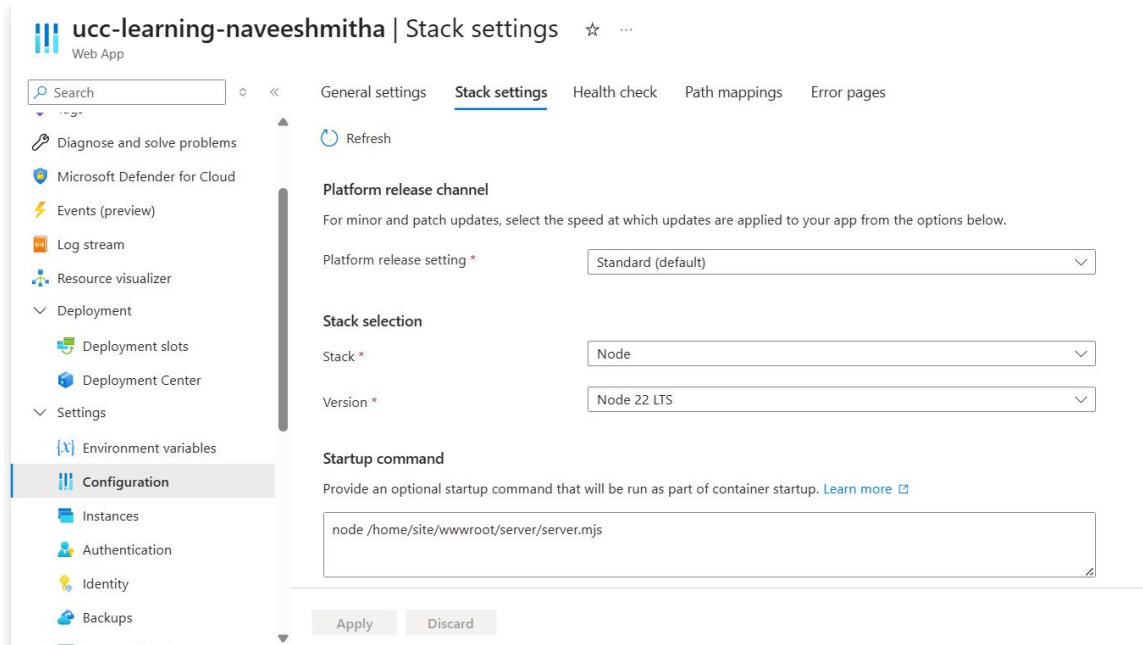
Error 1 — User Does Not Have Access / SCM Not Enabled

After setting up Deployment Center and saving, the deployment ran but failed. I got an error that said something like the user does not have access.

Error: Deployment failed. Something about user permissions or access denied. The deployment could not push to the app service.

After checking around I found out that for my Student subscription the SCM (Source Control Manager) access was disabled by default. SCM is what allows deployment tools and GitHub Actions to actually push files into your App Service.

Fix: Go to App Service > Settings > Configuration > General Settings. Find the option called SCM Basic Auth Publishing Credentials and turn it On. Also turn on FTP Basic Auth Publishing Credentials. Then save. This allows the deployment to authenticate and push files.



General Settings - enabling SCM and FTP Basic Auth publishing credentials

Error 2 — Still Getting Errors, Needed Publish Profile

Even after enabling SCM I was still getting errors. So then I used another method which is the Publish Profile approach.

In the App Service top toolbar there is a button called Get Publish Profile or Download Publish Profile. I clicked that and it downloaded an XML file with the credentials needed to deploy.

What I did: Downloaded the publish profile XML file from App Service. Opened the file and copied all the content. Then went to my GitHub repository > Settings > Secrets and Variables > Actions > New Repository Secret. Created a secret called AZURE_WEBAPP_PUBLISH_PROFILE and pasted the XML content as the value. Then committed the changes to the workflow YAML in the repo.

Why is this needed? GitHub Actions needs credentials to authenticate with Azure and push files. The publish profile contains those credentials in encrypted form. By storing it as a GitHub secret, the workflow can use it securely without exposing the credentials in the code.

Error 3 — Website Deployed But Showing Blank / Wrong Page

After fixing the credentials the deployment ran successfully but when I opened the URL my website was not showing. It was either blank or showing a wrong page.

Problem: Deployment succeeded but browsing the URL was not showing my Angular app. Just blank page or wrong content.

I realized the issue was with the package path in the GitHub Actions workflow. The workflow was pointing to the wrong folder when deploying. At first I thought it was the package.json location so I tried pointing to that but still nothing.

Then I checked my build output and found my Angular app builds to dist/uccbot folder. That is the actual folder with all the compiled files that need to be deployed. I updated the workflow to point to dist/uccbot.

Fix: In the GitHub Actions workflow YAML file, set the package path to dist/uccbot (or whatever your angular project output folder is). This tells the deployment to pick up only the built files, not the entire source code.

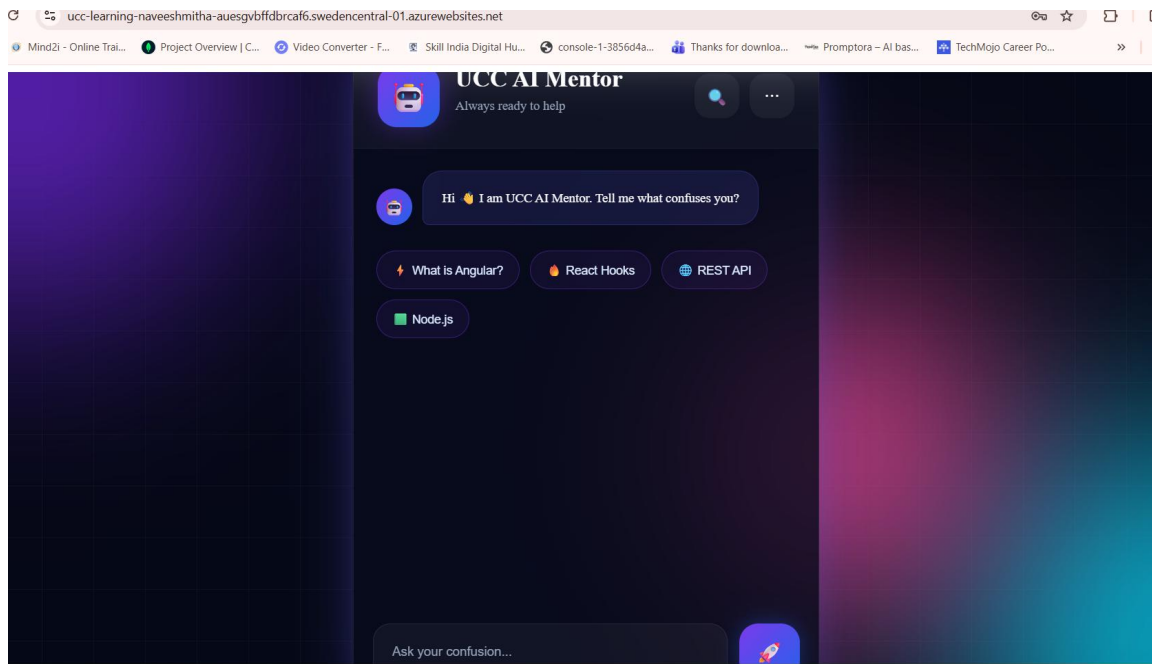
Error 4 — App Crashing on Startup, server.mjs Issue

Files were deploying correctly now but when I browsed the URL the app was crashing or not loading. The Azure logs showed it could not find how to start the application.

Problem: App deployed but crashing. Azure could not find the entry point to start the Node.js server.

My Angular project uses Server Side Rendering (SSR) so it has a Node.js server file called server.mjs inside the dist folder. Azure does not know automatically which file to run to start the app, so I had to tell it.

Fix: Go to App Service > Settings > Configuration > General Settings. Find the Startup Command field. I entered: node dist/uccbot/server/server.mjs and saved. This tells Azure exactly which file to run to start the application.



Configuration settings showing the startup command I added - server.mjs path

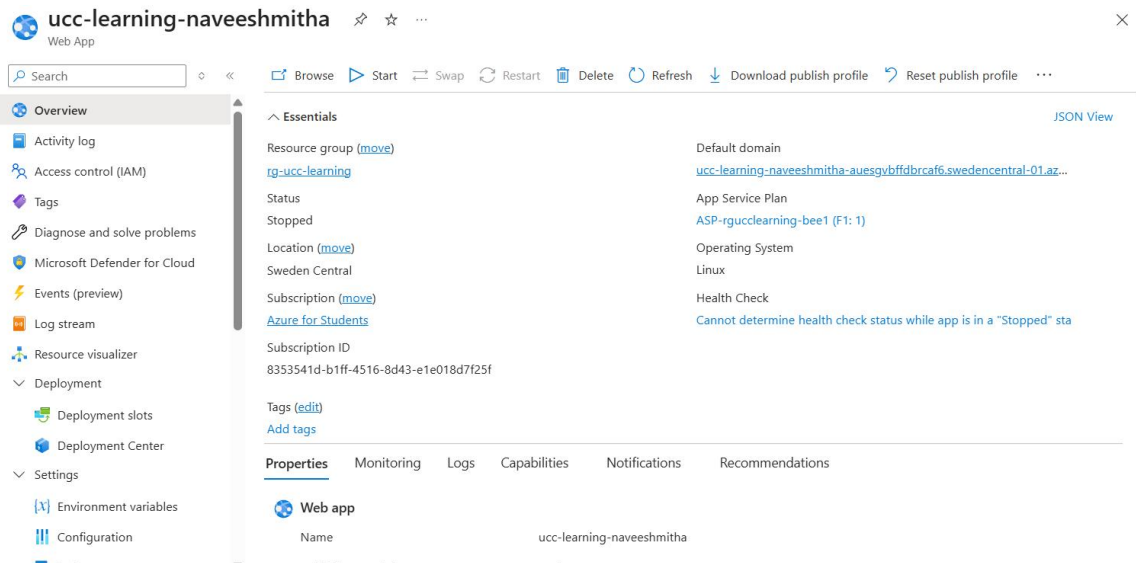
Step 10 — Finally My Website is Live!

After all those errors and fixes — enabling SCM, adding the publish profile secret, fixing the package path to dist/uccbot, and setting the startup command to server.mjs — I finally deployed successfully.

I opened the browser and went to the URL that Azure gave me:

```
https://ucc-learning-naveeshmitha-auesgybffdbrcaf6.swedencentral-01.azurewebsites.net/
```

And my Angular website loaded!! I was so happy after going through all those errors and finally getting it live. Each error taught me something and I now understand the whole deployment process properly.



My Angular website live on Azure! Finally done!

Step 11 — Stopping the Server

Once I confirmed the website was live, I stopped the App Service because I don't need it running 24/7 right now. In Azure Portal inside the App Service I clicked the Stop button at the top.

Important thing about Free F1 Plan: Here is one main thing I want to mention. When creating the Web App I selected the Free F1 plan. This means even if I don't stop the server and even if I don't delete the resource group, it will NOT take any of my Azure credits. The Free F1 plan is completely free. So don't worry about it eating up your credits.

But if you want to completely clean everything up then you can go to the Resource Group and delete it. Deleting the resource group will delete the App Service and everything else inside it in one go.

Quick Summary of Everything I Did

Here is the whole thing in short for reference:

- Logged into dev.azure.com
- Went to Pipelines and created a New Pipeline
- Connected GitHub and selected my Angular repo
- Selected Node.js with Angular template — got the YAML automatically
- Got budget error in angular.json — increased budget limits and committed to GitHub

- Pipeline built successfully — CI done
- Went to portal.azure.com (separate website)
- Created Resource Group: rg-uuc-learning
- Created App Service Web App with Node 22 LTS, Linux, Free F1, Sweden Central (region policy issue)
- Opened Deployment Center in the App Service, connected GitHub repo and branch
- Got access error — enabled SCM and FTP in General Settings
- Got auth error — downloaded publish profile, added it as GitHub Actions secret
- Website deployed but blank — fixed package path to dist/uccbot
- App crashing — added startup command: node dist/uccbot/server/server.mjs
- Website finally live!
- Stopped the server (Free F1 means no credits used either way)

That's it. Took a lot of trial and error but now I know exactly how this works.